RTL Design Sherpa

RTL AMBA AXI5-Stream Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXIS5 Master with Clock Gating

Module: `axis5_master_cg.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

1.1 Overview

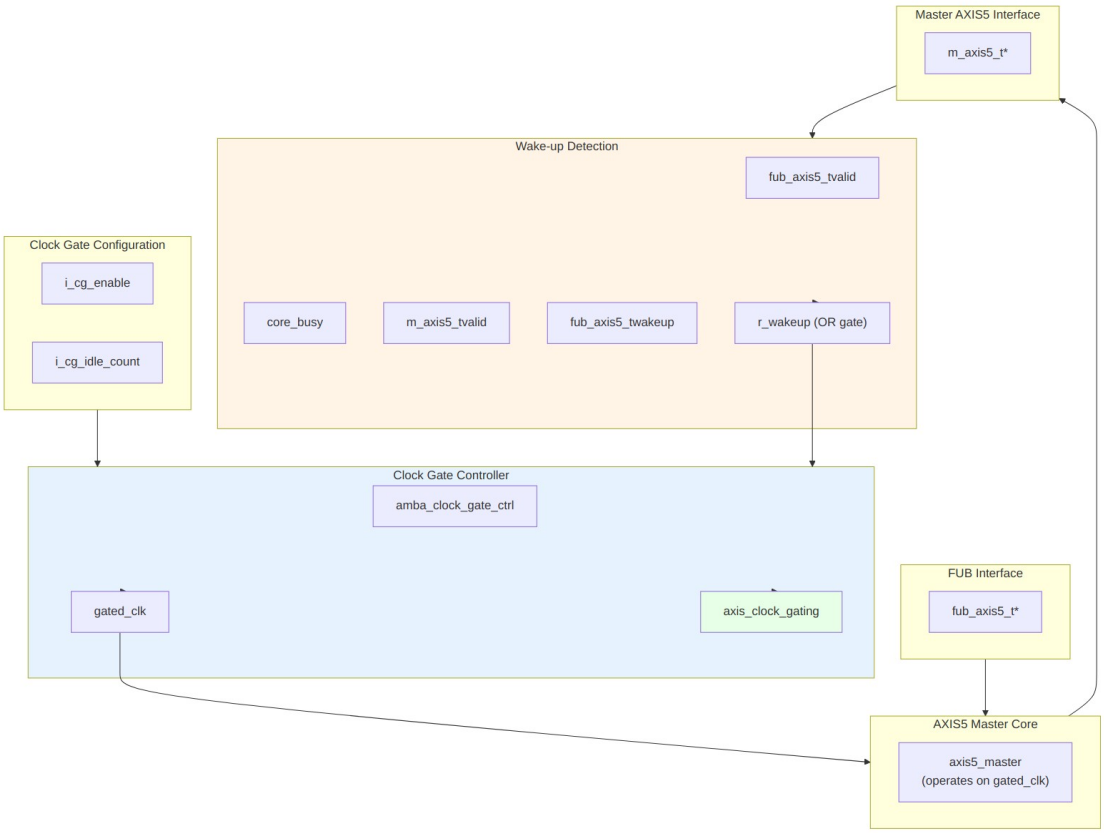
The AXIS5 Master CG (Clock Gated) module implements an AXI5-Stream master interface with integrated clock gating for power savings. It wraps the standard AXIS5 master with the AMBA clock gate controller, automatically gating the clock during idle periods while preserving full AXIS5 functionality including wake-up signaling and optional parity.

1.1.1 Key Features

- Full AXI5-Stream protocol compliance with AMBA5 extensions
- Automatic clock gating during idle periods for power savings
- TWAKEUP: Wake-up signaling integration with clock gate control
- TPARITY: Optional parity protection (1 bit per byte)
- Configurable idle count threshold for gating activation
- Internal skid buffer for backpressure handling
- Parity error detection and reporting
- Clock gating status output

1.1.2 Power Management Features

Feature	Implementation	Benefit
Clock gating	Automatic via idle detection	Reduces dynamic power
Wake-up integration	TWAKEUP prevents gating	Preserves responsiveness
Configurable threshold	Programmable idle count	Tune power vs. latency
Activity detection	Multi-source OR logic	Comprehensive coverage



Module Architecture

1.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable TPARITY signal (1=enabled)
CG_IDLE_COUNT_WIDTH	int	4	Clock gate idle counter width
DW	int	AXIS_DATA_WIDTH	Data width short name (calculated)
IW	int	AXIS_ID_WIDTH	ID width short name (calculated)
DESTW	int	AXIS_DEST_WIDTH	DEST width short name (calculated)
UW	int	AXIS_USER_WIDTH	USER width short name (calculated)
SW	int	DW/8	Strobe width in bytes (calculated)
PW	int	SW	Parity width - 1 bit per byte (calculated)
ICW	int	CG_IDLE_COUNT_WIDTH	Idle count width short name (calculated)

1.3 Ports

1.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS ungated clock (always running)
aresetn	1	Input	AXIS active-low asynchronous reset

1.3.2 Clock Gating Configuration

Port	Width	Direction	Description
i_cg_enable	1	Input	Clock gating enable (1=allow gating, 0=force always on)
i_cg_idle_count	ICW	Input	Idle clock cycles before gating activates

1.3.3 FUB AXIS5 Interface (Input Side)

Port	Width	Direction	Description
fub_axis5_tdata	DW	Input	Transfer data
fub_axis5_tstrb	SW	Input	Transfer byte strobes
fub_axis5_tlast	1	Input	Last transfer in packet
fub_axis5_tid	IW_WIDTH	Input	Transfer ID (optional)
fub_axis5_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
fub_axis5_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
fub_axis5_t	1	Input	Transfer valid

Port	Width	Direction	Description
valid			
fub_axis5_tready	1	Output	Transfer ready (skid buffer not full)
fub_axis5_twakeup	1	Input	Wake-up signal (prevents clock gating)
fub_axis5_tparity	PW_WIDTH	Input	Data parity per byte (AXIS5 extension)

1.3.4 Master AXIS5 Interface (Output Side)

Port	Width	Direction	Description
m_axis5_tdata	DW	Output	Transfer data
m_axis5_tstrb	SW	Output	Transfer byte strobes
m_axis5_tlast	1	Output	Last transfer in packet
m_axis5_tid	IW_WIDTH	Output	Transfer ID (optional)
m_axis5_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
m_axis5_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
m_axis5_tvalid	1	Output	Transfer valid
m_axis5_tready	1	Input	Transfer ready from downstream
m_axis5_twakeup	1	Output	Wake-up signal
m_axis5_tparity	PW_WIDTH	Output	Data parity per byte (AXIS5 extension)

1.3.5 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in

Port	Width	Direction	Description
			buffer or input valid)
parity_error	1	Output	Parity error detected (sticky flag)
axis_clock_gating	1	Output	Clock gating active status (1=gated, 0=running)

1.4 Functionality

1.4.1 Clock Gating Control

Wake-up detection logic:

```
r_wakeup <= fub_axis5_tvalid ||  // Input has data
            core_busy ||          // Core processing
            m_axis5_tvalid ||     // Output has data
            fub_axis5_twakeup;    // Explicit wake-up
```

The clock gate controller: 1. Monitors `r_wakeup` for activity 2. If idle (r_wakeup=0) for `i_cg_idle_count` cycles, gates clock 3. On activity (r_wakeup=1), immediately ungates clock 4. Respects `i_cg_enable` (forced on if disabled)

1.4.2 Power Saving Mechanism

Clock gating states:

Condition	r_wakeup	Clock State	Power State
Active transfer	1	Running	Full power
Buffer has data	1	Running	Full power
Wake-up asserted	1	Running	Full power
Idle < threshold	0	Running	Full power
Idle ≥ threshold	0	Gated	Low power

Benefits: - Reduces dynamic power during idle periods - Zero latency wake-up on new transfers - No protocol impact (transparent to upstream/downstream)

1.4.3 Integration with AXIS5 Extensions

Wake-up signal dual purpose: 1. **Protocol:** Power management hint to downstream 2. **Clock gating:** Prevents local clock gating

This ensures wake-up events are never missed due to clock gating.

1.4.4 Idle Count Configuration

Typical values for `i_cg_idle_count`: - **Aggressive power saving:** 2-4 cycles - Quick gating, may have frequent gate/ungate transitions - Best for bursty traffic with long idle periods - **Balanced:** 8-16 cycles - Reduces gate/ungate overhead - Good for moderate traffic patterns - **Conservative:** 32-64 cycles - Only gates during extended idle - Minimal gate/ungate overhead

Trade-off: Lower count = more power savings but more gate transitions (dynamic overhead)

1.5 Timing Diagrams

1.5.1 Clock Gating Activation

```
TODO: Wavedrom timing diagram showing:
- aclk (always running)
- r_wakeup (activity detection)
- i_cg_idle_count (threshold)
- gated_clk (starts gating after threshold)
- axis_clock_gating (status)
- fub_axis5_tvalid (new transfer wakes up)
```

1.5.2 Wake-up Signal Preventing Gating

```
TODO: Wavedrom timing diagram showing:
- aclk
- fub_axis5_twakeup (asserted)
- r_wakeup (stays high)
- gated_clk (remains running)
- axis_clock_gating (stays 0)
```

1.5.3 Transfer During Idle Period

```
TODO: Wavedrom timing diagram showing:
- aclk
- gated_clk (gated, then ungates on transfer)
- fub_axis5_tvalid (new transfer)
- r_wakeup (0 → 1 transition)
- axis_clock_gating (1 → 0)
- m_axis5_tvalid (output after ungate)
```

1.6 Usage Example

1.6.1 Basic Clock-Gated Configuration

```
axis5_master_cg #(
    .SKID_DEPTH             (4),
    .AXIS_DATA_WIDTH        (64),
    .AXIS_ID_WIDTH          (8),
    .AXIS_DEST_WIDTH        (4),
    .AXIS_USER_WIDTH        (1),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (0),
    .CG_IDLE_COUNT_WIDTH    (4)
) u_axis5_master_cg (
    .aclk                   (axis_clk),
    .aresetn                (axis_rst_n),

    // Clock gating configuration
    .i_cg_enable            (1'b1),         // Enable clock gating
    .i_cg_idle_count        (4'd8),         // Gate after 8 idle cycles

    // FUB interface (from upstream)
    .fub_axis5_tdata        (fub_tdata),
    .fub_axis5_tstrb        (fub_tstrb),
    .fub_axis5_tlast        (fub_tlast),
    .fub_axis5_tid          (fub_tid),
    .fub_axis5_tdest        (fub_tdest),
    .fub_axis5_tuser        (fub_tuser),
    .fub_axis5_tvalid       (fub_tvalid),
    .fub_axis5_tready       (fub_tready),
    .fub_axis5_twakeup      (fub_twakeup),
    .fub_axis5_tparity      (8'h00),

    // Master AXIS5 interface (to downstream)
    .m_axis5_tdata          (m_axis_tdata),
    .m_axis5_tstrb          (m_axis_tstrb),
    .m_axis5_tlast          (m_axis_tlast),
    .m_axis5_tid            (m_axis_tid),
    .m_axis5_tdest          (m_axis_tdest),
    .m_axis5_tuser          (m_axis_tuser),
    .m_axis5_tvalid         (m_axis_tvalid),
    .m_axis5_tready         (m_axis_tready),
    .m_axis5_twakeup        (m_axis_twakeup),
    .m_axis5_tparity        (),

    // Status
    .busy                   (axis_busy),
```

```
    .parity_error           (),
    .axis_clock_gating      (axis_clk_gated)  // Monitor gating status
);
```

1.6.2 Dynamic Clock Gating Control

```
// Runtime control of clock gating
logic cg_enable;
logic [3:0] cg_idle_count;

axis5_master_cg #(
    .AXIS_DATA_WIDTH        (32),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (1),
    .CG_IDLE_COUNT_WIDTH    (4)
) u_axis5_master_cg (
    .aclk                   (sys_clk),
    .aresetn                (sys_rst_n),

    // Dynamic configuration via registers
    .i_cg_enable            (cg_enable),
    .i_cg_idle_count        (cg_idle_count),

    // ... ports ...

    .busy                   (axis_busy),
    .parity_error           (parity_err),
    .axis_clock_gating      (clk_gated_status)
);

// Configuration register interface
always_ff @(posedge cfg_clk or negedge cfg_rst_n) begin
    if (!cfg_rst_n) begin
        cg_enable <= 1'b1;      // Default: gating enabled
        cg_idle_count <= 4'd16; // Default: 16 cycle threshold
    end else if (cfg_write) begin
        case (cfg_addr)
            ADDR_CG_CTRL: cg_enable <= cfg_wdata[0];
            ADDR_CG_IDLE: cg_idle_count <= cfg_wdata[3:0];
        endcase
    end
end

// Monitor power savings
always_ff @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n)
        gated_cycle_count <= '0;
```

```
    else if (clk_gated_status)
        gated_cycle_count <= gated_cycle_count + 1;
end
```

1.6.3 Power Profiling Example

```
// Instantiate both gated and non-gated versions for comparison
axis5_master_cg #(
    .AXIS_DATA_WIDTH    (64),
    .CG_IDLE_COUNT_WIDTH(4)
) u_gated (
    .aclk               (clk),
    .aresetn            (rst_n),
    .i_cg_enable        (1'b1),
    .i_cg_idle_count    (4'd8),
    // ... ports ...
    .axis_clock_gating  (gated_status)
);

// Track gating statistics
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        total_cycles <= '0;
        gated_cycles <= '0;
    end else begin
        total_cycles <= total_cycles + 1;
        if (gated_status)
            gated_cycles <= gated_cycles + 1;
    end
end

// Calculate power savings percentage
assign power_savings_pct = (gated_cycles * 100) / total_cycles;
```

1.7 Design Notes

1.7.1 Clock Gating vs. Non-Gated Variant

Aspect	axis5_master	axis5_master_cg
Area	Smaller	+5-10% (clock gate logic)
Dynamic power	Higher	Lower (gating reduces)
Static power	Same	Same

Aspect	axis5_master	axis5_master_cg
Latency	Lower	Same (zero-cycle wake-up)
Complexity	Lower	Higher (gating control)
Use case	Always-on systems	Power-sensitive designs

When to use clock-gated variant: - Battery-powered devices - Bursty traffic patterns with idle periods - Power budget constraints - SoC integration with power domains

1.7.2 Wake-up Integration Benefits

TWAKEUP integration with clock gating provides: 1. **Event preservation:** Wake-up events prevent clock gating 2. **Zero latency:** Immediate clock activation 3. **Protocol compliance:** Wake-up propagated correctly 4. **Power efficiency:** Only wake when necessary

1.7.3 Gate Transition Overhead

Each clock gate transition has overhead: - **Gate activation:** 1-2 cycles to fully gate - **Ungate activation:** 0-1 cycles to restore clock - **Dynamic power:** Gate/ungate switching consumes power

Recommendation: Set `i_cg_idle_count` high enough to amortize transition overhead.

1.7.4 Clock Domain Considerations

- **Input signals:** Must be on `aclk` domain (always running)
- **Core logic:** Operates on `gated_clk` domain
- **Output signals:** Driven by `gated_clk` domain
- **Status outputs:** `axis_clock_gating` on `aclk` domain

No explicit CDC (clock domain crossing) needed - gated_clk is derived from aclk.

1.8 Related Documentation

- **AXIS5 Master** - Base AXIS5 master without clock gating
- **AXIS5 Slave CG** - Clock-gated slave variant
- **AXIS5 Slave** - Base AXIS5 slave
- **AMBA Clock Gate Controller** - Clock gating controller
- **AMBA5 Power Management** - AMBA5 power features

1.9 Navigation

- ← Back to AXIS5 Index
- ← Back to RTLAmba Index
- ← Back to Main Documentation Index

2 AXIS5 Master

Module: `axis5_master.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

2.1 Overview

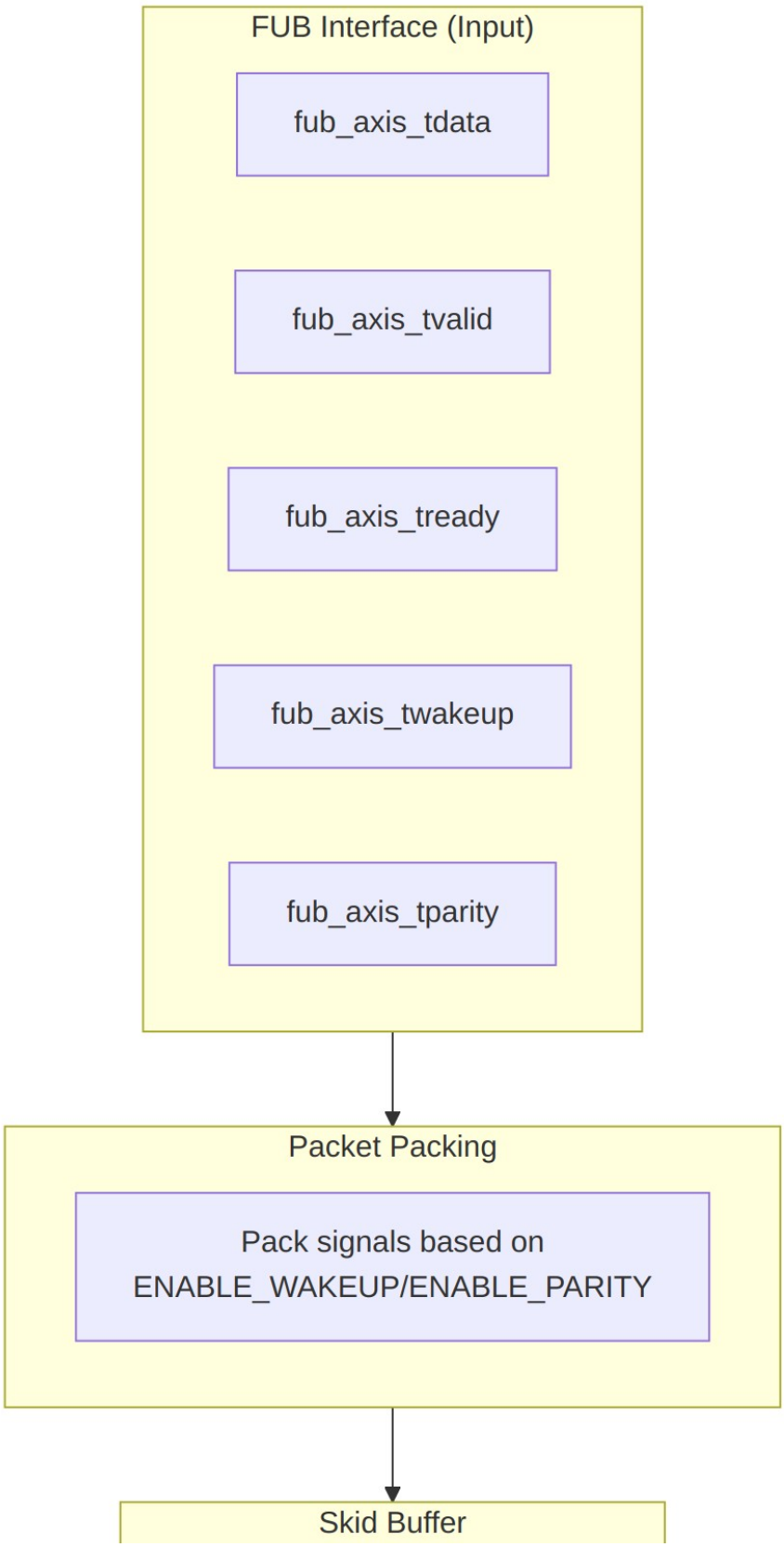
The AXIS5 Master module implements an AXI5-Stream master interface with AMBA5 extensions including wake-up signaling for power management and optional parity for data integrity. It provides buffering through an internal skid buffer for improved system performance.

2.1.1 Key Features

- Full AXI5-Stream protocol compliance
- TWAKEUP: Wake-up signaling for power management
- TPARITY: Optional parity protection (1 bit per byte)
- Internal skid buffer for backpressure handling
- Configurable data, ID, destination, and user signal widths
- Parity error detection and reporting
- Busy status indication

2.1.2 AXIS5 Extensions Over AXIS4

Feature	AXIS4	AXIS5
Wake-up signal	None	TWAKEUP (configurable)
Data parity	None	TPARITY (1 bit per byte, configurable)
Parity error detection	None	Built-in error flag


FUB Interface (Input)
fub_axis_tdata
fub_axis_tvalid
fub_axis_tready
fub_axis_twakeup
fub_axis_tparity
Packet Packing
Pack signals based on
ENABLE_WAKEUP/ENABLE_PARITY
Skid Buffer

2.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable TPARITY signal (1=enabled)
DW	int	AXIS_DATA_WIDTH	Data width short name (calculated)
IW	int	AXIS_ID_WIDTH	ID width short name (calculated)
DESTW	int	AXIS_DEST_WIDTH	DEST width short name (calculated)
UW	int	AXIS_USER_WIDTH	USER width short name (calculated)
SW	int	DW/8	Strobe width in bytes (calculated)
PW	int	SW	Parity width - 1 bit per byte (calculated)

2.3 Ports

2.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS clock
aresetn	1	Input	AXIS active-low asynchronous reset

2.3.2 FUB AXIS5 Interface (Input Side)

Port	Width	Direction	Description
fub_axis_tdata	DW	Input	Transfer data
fub_axis_tstrb	SW	Input	Transfer byte strobes
fub_axis_tlast	1	Input	Last transfer in packet
fub_axis_tid	IW_WIDTH	Input	Transfer ID (optional)
fub_axis_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
fub_axis_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
fub_axis_tvalid	1	Input	Transfer valid
fub_axis_tready	1	Output	Transfer ready (skid buffer not full)
fub_axis_twakeup	1	Input	Wake-up signal (AXIS5 extension)
fub_axis_tparity	PW_WIDTH	Input	Data parity per byte (AXIS5 extension)

2.3.3 Master AXIS5 Interface (Output Side)

Port	Width	Direction	Description
m_axis_tdata	DW	Output	Transfer data
m_axis_tstrb	SW	Output	Transfer byte strobes
m_axis_tlast	1	Output	Last transfer in packet
m_axis_tid	IW_WIDTH	Output	Transfer ID (optional)
m_axis_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
m_axis_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
m_axis_tvalid	1	Output	Transfer valid
m_axis_tready	1	Input	Transfer ready from downstream
m_axis_twakeup	1	Output	Wake-up signal (AXIS5 extension)
m_axis_tparity	PW_WIDTH	Output	Data parity per byte (AXIS5 extension)

2.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Parity error detected (sticky flag)

2.4 Functionality

2.4.1 Skid Buffer Operation

The module uses an internal `gaxi_skid_buffer` to: - Accept incoming transfers even when downstream is not ready - Prevent backpressure propagation - Provide registered outputs for timing closure - Track buffer occupancy via `busy` signal

2.4.2 Packet Packing/Unpacking

Conditional packing based on configuration:

```
// Full feature set (ENABLE_WAKEUP=1, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup, tparity}

// Wake-up only (ENABLE_WAKEUP=1, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup}

// Parity only (ENABLE_WAKEUP=0, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, tparity}

// Base AXIS4 (ENABLE_WAKEUP=0, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser}
```

2.4.3 Parity Checking (Optional)

When `ENABLE_PARITY=1`: 1. Calculate odd parity for each data byte: `parity[i] = ^tdata[i*8 +: 8]` 2. Compare calculated parity with received `m_axis_tparity` 3. Set `parity_error` flag on mismatch (sticky, cleared by reset) 4. Parity check occurs on valid output transfers

2.4.4 Busy Signal

The `busy` output indicates: - Input side has valid data (`fub_axis_tvalid`) - Skid buffer contains data (`int_t_count > 0`)

2.5 Timing Diagrams

2.5.1 Basic Transfer with Wake-up

```
TODO: Wavedrom timing diagram showing:
- aclk
- fub_axis_tvalid/tready
- fub_axis_tdata
- fub_axis_tlast
- fub_axis_twakeup (AXIS5 extension)
- m_axis_tvalid/tready
```

```
- m_axis_tdata
- m_axis_twakeup
- busy
```

2.5.2 Transfer with Parity Error

```
TODO: Wavedrom timing diagram showing:
- aclk
- m_axis_tdata
- m_axis_tparity (received)
- calculated_parity
- parity_mismatch
- parity_error (sticky flag)
```

2.5.3 Skid Buffer Backpressure

```
TODO: Wavedrom timing diagram showing:
- aclk
- fub_axis_tvalid/tready
- m_axis_tvalid/tready (downstream blocked)
- int_t_count (buffer fill level)
- busy
```

2.6 Usage Example

2.6.1 Basic Configuration

```
axis5_master #(
    .SKID_DEPTH       (4),
    .AXIS_DATA_WIDTH  (64),
    .AXIS_ID_WIDTH    (8),
    .AXIS_DEST_WIDTH  (4),
    .AXIS_USER_WIDTH  (1),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (0)
) u_axis5_master (
    .aclk               (axis_clk),
    .aresetn            (axis_rst_n),

    // FUB interface (from upstream)
    .fub_axis_tdata     (fub_tdata),
    .fub_axis_tstrb     (fub_tstrb),
    .fub_axis_tlast     (fub_tlast),
    .fub_axis_tid       (fub_tid),
    .fub_axis_tdest     (fub_tdest),
    .fub_axis_tuser     (fub_tuser),
    .fub_axis_tvalid    (fub_tvalid),
```

```
    .fub_axis_tready     (fub_tready),
    .fub_axis_twakeup    (fub_twakeup),
    .fub_axis_tparity    (8'h00),  // Not used when ENABLE_PARITY=0

    // Master AXIS5 interface (to downstream)
    .m_axis_tdata        (m_axis_tdata),
    .m_axis_tstrb        (m_axis_tstrb),
    .m_axis_tlast        (m_axis_tlast),
    .m_axis_tid          (m_axis_tid),
    .m_axis_tdest        (m_axis_tdest),
    .m_axis_tuser        (m_axis_tuser),
    .m_axis_tvalid       (m_axis_tvalid),
    .m_axis_tready       (m_axis_tready),
    .m_axis_twakeup      (m_axis_twakeup),
    .m_axis_tparity      (),  // Not used when ENABLE_PARITY=0

    // Status
    .busy                (axis_busy),
    .parity_error        ()  // Not used when ENABLE_PARITY=0
);
```

2.6.2 With Parity Protection

```
axis5_master #(
    .AXIS_DATA_WIDTH  (32),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (1)  // Enable parity checking
) u_axis5_master_parity (
    .aclk                (axis_clk),
    .aresetn             (axis_rst_n),

    // FUB interface with parity
    .fub_axis_tdata      (fub_tdata),
    .fub_axis_tstrb      (fub_tstrb),
    .fub_axis_tlast      (fub_tlast),
    .fub_axis_tid        (fub_tid),
    .fub_axis_tdest      (fub_tdest),
    .fub_axis_tuser      (fub_tuser),
    .fub_axis_tvalid     (fub_tvalid),
    .fub_axis_tready     (fub_tready),
    .fub_axis_twakeup    (fub_twakeup),
    .fub_axis_tparity    (fub_tparity),  // 4 bits for 32-bit data

    // Master AXIS5 interface
    .m_axis_tdata        (m_axis_tdata),
    .m_axis_tstrb        (m_axis_tstrb),
    .m_axis_tlast        (m_axis_tlast),
    .m_axis_tid          (m_axis_tid),
```

```
    .m_axis_tdest        (m_axis_tdest),
    .m_axis_tuser        (m_axis_tuser),
    .m_axis_tvalid       (m_axis_tvalid),
    .m_axis_tready       (m_axis_tready),
    .m_axis_twakeup      (m_axis_twakeup),
    .m_axis_tparity      (m_axis_tparity),

    // Status - monitor parity errors
    .busy                (axis_busy),
    .parity_error        (axis_parity_err)  // Sticky error flag
);

// Error handling
always_ff @(posedge axis_clk or negedge axis_rst_n) begin
    if (!axis_rst_n)
        error_count <= '0;
    else if (axis_parity_err && !prev_parity_err)
        error_count <= error_count + 1;
end
```

2.7 Design Notes

2.7.1 AXIS5 vs AXIS4 Differences

Feature	AXIS4	AXIS5
Wake-up signal	Not supported	TWAKEUP (optional)
Data parity	Not supported	TPARITY per byte (optional)
Power management	Limited	Enhanced via TWAKEUP
Data integrity	CRC/checksum in TUSER	Built-in parity option

2.7.2 Skid Buffer Sizing

- **Typical depth:** 4-8 entries
- Deeper buffers:
  - Absorb longer downstream stalls
  - Increase area and latency
- Shallower buffers:
  - Lower latency

- May cause upstream backpressure

Recommendation: Use depth 4 for most applications, increase if downstream frequently stalls.

2.7.3 Parity Implementation

When `ENABLE_PARITY=1`: - **Overhead:** 1 bit per data byte (12.5% for 8-bit bytes) - **Detection:** Single-bit errors only (odd parity) - **Correction:** None - error flag only - **Use case:** Low-cost error detection in reliable environments

For stronger protection, use: - CRC in TUSER field - ECC (external module) - End-to-end checksums at packet level

2.7.4 Optional Signal Widths

Setting width parameters to 0 disables optional signals: - `AXIS_ID_WIDTH=0` → TID tied to 0, saves area - `AXIS_DEST_WIDTH=0` → TDEST tied to 0 - `AXIS_USER_WIDTH=0` → TUSER tied to 0

Internal logic uses `IW_WIDTH = (IW > 0) ? IW : 1` to avoid zero-width signals.

2.8 Related Documentation

- **AXIS5 Slave** - AXIS5 slave interface
- **AXIS5 Master CG** - Clock-gated variant with power management
- **AXIS5 Slave CG** - Clock-gated slave variant
- **AXIS4 Master** - AXIS4 version for comparison
- **AMBA5 Overview** - AMBA5 specifications and extensions

2.9 Navigation

- **← Back to AXIS5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

3 AXIS5 Slave with Clock Gating

Module: `axis5_slave_cg.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

3.1 Overview

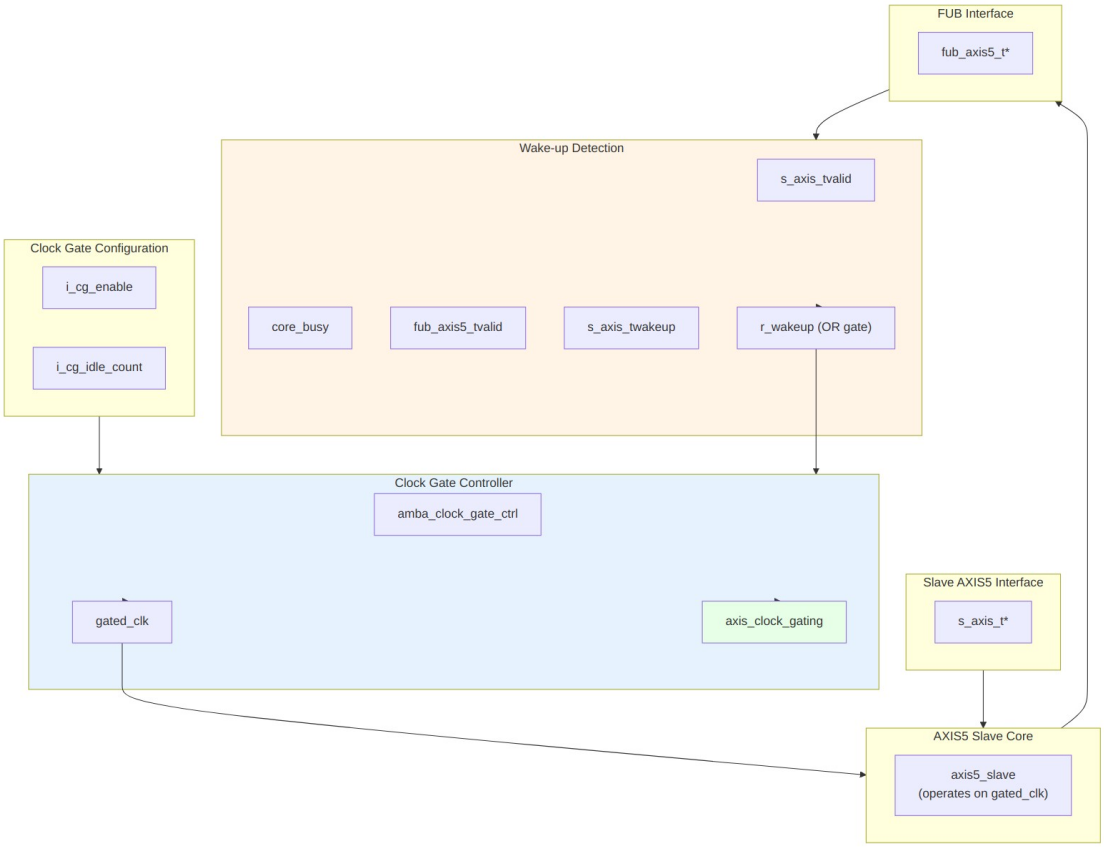
The AXIS5 Slave CG (Clock Gated) module implements an AXI5-Stream slave interface with integrated clock gating for power savings. It wraps the standard AXIS5 slave with the AMBA clock gate controller, automatically gating the clock during idle periods while preserving full AXIS5 functionality including wake-up signaling and optional parity.

3.1.1 Key Features

- Full AXI5-Stream protocol compliance with AMBA5 extensions
- Automatic clock gating during idle periods for power savings
- TWAKEUP: Wake-up signaling integration with clock gate control
- TPARITY: Optional parity protection (1 bit per byte) with error detection
- Configurable idle count threshold for gating activation
- Internal skid buffer for backpressure handling
- Parity error detection on input side
- Clock gating status output

3.1.2 Power Management Features

Feature	Implementation	Benefit
Clock gating	Automatic via idle detection	Reduces dynamic power
Wake-up integration	TWAKEUP prevents gating	Preserves responsiveness
Configurable threshold	Programmable idle count	Tune power vs. latency
Activity detection	Multi-source OR logic	Comprehensive coverage



Module Architecture

3.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)

Parameter	Type	Default	Description
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable TPARITY signal (1=enabled)
CG_IDLE_COUNT_WIDTH	int	4	Clock gate idle counter width
DW	int	AXIS_DATA_WIDTH	Data width short name (calculated)
IW	int	AXIS_ID_WIDTH	ID width short name (calculated)
DESTW	int	AXIS_DEST_WIDTH	DEST width short name (calculated)
UW	int	AXIS_USER_WIDTH	USER width short name (calculated)
SW	int	DW/8	Strobe width in bytes (calculated)
PW	int	SW	Parity width - 1 bit per byte (calculated)
ICW	int	CG_IDLE_COUNT_WIDTH	Idle count width short name (calculated)

3.3 Ports

3.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS ungated clock (always running)
aresetn	1	Input	AXIS active-low asynchronous reset

3.3.2 Clock Gating Configuration

Port	Width	Direction	Description
i_cg_enable	1	Input	Clock gating enable (1=allow gating, 0=force always on)
i_cg_idle_count	ICW	Input	Idle clock cycles before gating activates

3.3.3 Slave AXIS5 Interface (Input Side)

Port	Width	Direction	Description
s_axis_tdata	DW	Input	Transfer data from upstream
s_axis_tstrb	SW	Input	Transfer byte strobes
s_axis_tlast	1	Input	Last transfer in packet
s_axis_tid	IW_WIDTH	Input	Transfer ID (optional)
s_axis_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
s_axis_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
s_axis_tvalid	1	Input	Transfer valid from upstream
s_axis_tready	1	Output	Transfer ready (skid buffer not full)
s_axis_twakeup	1	Input	Wake-up signal (prevents clock gating)
s_axis_tparity	PW_WIDTH	Input	Data parity per byte (AXIS5 extension)

3.3.4 FUB AXIS5 Interface (Output Side to Backend)

Port	Width	Direction	Description
fub_axis5_tdata	DW	Output	Transfer data to backend

Port	Width	Direction	Description
fub_axis5_tstrb	SW	Output	Transfer byte strobes
fub_axis5_tlast	1	Output	Last transfer in packet
fub_axis5_tid	IW_WIDTH	Output	Transfer ID (optional)
fub_axis5_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
fub_axis5_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
fub_axis5_tvalid	1	Output	Transfer valid to backend
fub_axis5_tready	1	Input	Transfer ready from backend
fub_axis5_twakeup	1	Output	Wake-up signal
fub_axis5_tparity	PW_WIDTH	Output	Data parity per byte (AXIS5 extension)

3.3.5 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Parity error detected on input (sticky flag)
axis_clock_gating	1	Output	Clock gating active status (1=gated, 0=running)

3.4 Functionality

3.4.1 Clock Gating Control

Wake-up detection logic:

```
r_wakeup <= s_axis_tvalid ||    // Input has data
            core_busy ||         // Core processing
            fub_axis5_tvalid ||  // Backend has data
            s_axis_twakeup;      // Explicit wake-up
```

The clock gate controller: 1. Monitors `r_wakeup` for activity 2. If idle (r_wakeup=0) for `i_cg_idle_count` cycles, gates clock 3. On activity (r_wakeup=1), immediately ungates clock 4. Respects `i_cg_enable` (forced on if disabled)

3.4.2 Power Saving Mechanism

Clock gating states:

Condition	r_wakeup	Clock State	Power State
Receiving data	1	Running	Full power
Buffer has data	1	Running	Full power
Backend processing	1	Running	Full power
Wake-up asserted	1	Running	Full power
Idle < threshold	0	Running	Full power
Idle ≥ threshold	0	Gated	Low power

Benefits: - Reduces dynamic power during idle periods - Zero latency wake-up on new transfers - No protocol impact (transparent to upstream/backend)

3.4.3 Parity Error Detection

When `ENABLE_PARITY=1`: - Parity checked on **input side** (`s_axis_tdata` vs `s_axis_tparity`) - Early error detection at receive interface - `parity_error` flag is sticky (latches until reset) - Error checking continues even during clock gating transitions

3.4.4 Idle Count Configuration

Typical values for `i_cg_idle_count`: - **Aggressive power saving:** 2-4 cycles - Quick gating, may have frequent gate/ungate transitions - Best for bursty traffic with long idle periods - **Balanced:** 8-16 cycles - Reduces gate/ungate overhead - Good for moderate traffic patterns - **Conservative:** 32-64 cycles - Only gates during extended idle - Minimal gate/ungate overhead

Trade-off: Lower count = more power savings but more gate transitions (dynamic overhead)

3.5 Timing Diagrams

3.5.1 Clock Gating Activation on Slave

TODO: Wavedrom timing diagram showing:
- aclk (always running)
- s_axis_tvalid (input activity)
- r_wakeup (activity detection)
- i_cg_idle_count (threshold)
- gated_clk (starts gating after threshold)
- axis_clock_gating (status)
- New s_axis_tvalid arrival ungates clock

3.5.2 Wake-up Preventing Gating During Receive

TODO: Wavedrom timing diagram showing:
- aclk
- s_axis_twakeup (asserted during transfer)
- s_axis_tvalid
- r_wakeup (stays high)
- gated_clk (remains running)
- axis_clock_gating (stays 0)
- fub_axis5_tvalid (output forwarded)

3.5.3 Backend Backpressure with Clock Gating

TODO: Wavedrom timing diagram showing:
- aclk
- s_axis_tvalid (upstream sending)
- fub_axis5_tready (backend blocked)
- core_busy (stays high)
- r_wakeup (stays high)
- gated_clk (remains running due to busy)
- axis_clock_gating (stays 0 during backpressure)

3.6 Usage Example

3.6.1 Basic Clock-Gated Slave

```
axis5_slave_cg #(
    .SKID_DEPTH             (4),
    .AXIS_DATA_WIDTH        (64),
    .AXIS_ID_WIDTH          (8),
    .AXIS_DEST_WIDTH        (4),
    .AXIS_USER_WIDTH        (1),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (0),
    .CG_IDLE_COUNT_WIDTH    (4)
```

```
) u_axis5_slave_cg (
    .aclk                   (axis_clk),
    .aresetn                (axis_rst_n),

    // Clock gating configuration
    .i_cg_enable            (1'b1),        // Enable clock gating
    .i_cg_idle_count        (4'd8),        // Gate after 8 idle cycles

    // Slave interface (from upstream)
    .s_axis_tdata           (s_axis_tdata),
    .s_axis_tstrb           (s_axis_tstrb),
    .s_axis_tlast           (s_axis_tlast),
    .s_axis_tid             (s_axis_tid),
    .s_axis_tdest           (s_axis_tdest),
    .s_axis_tuser           (s_axis_tuser),
    .s_axis_tvalid          (s_axis_tvalid),
    .s_axis_tready          (s_axis_tready),
    .s_axis_twakeup         (s_axis_twakeup),
    .s_axis_tparity         (8'h00),

    // FUB interface (to backend)
    .fub_axis5_tdata        (fub_tdata),
    .fub_axis5_tstrb        (fub_tstrb),
    .fub_axis5_tlast        (fub_tlast),
    .fub_axis5_tid          (fub_tid),
    .fub_axis5_tdest        (fub_tdest),
    .fub_axis5_tuser        (fub_tuser),
    .fub_axis5_tvalid       (fub_tvalid),
    .fub_axis5_tready       (fub_tready),
    .fub_axis5_twakeup      (fub_twakeup),
    .fub_axis5_tparity      (),

    // Status
    .busy                   (axis_busy),
    .parity_error           (),
    .axis_clock_gating      (axis_clk_gated)  // Monitor gating status
);
```

3.6.2 With Parity Protection and Power Monitoring

```
// Clock-gated slave with parity and comprehensive monitoring
axis5_slave_cg #(
    .AXIS_DATA_WIDTH        (32),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (1),  // Enable parity checking
    .CG_IDLE_COUNT_WIDTH    (4)
) u_rx_slave_cg (
    .aclk                   (sys_clk),
```

```
    .aresetn                (sys_rst_n),

    // Dynamic clock gating control
    .i_cg_enable            (cfg_cg_enable),
    .i_cg_idle_count        (cfg_cg_threshold),

    // Slave interface with parity
    .s_axis_tdata           (rx_tdata),
    .s_axis_tstrb           (rx_tstrb),
    .s_axis_tlast           (rx_tlast),
    .s_axis_tid             (rx_tid),
    .s_axis_tdest           (rx_tdest),
    .s_axis_tuser           (rx_tuser),
    .s_axis_tvalid          (rx_tvalid),
    .s_axis_tready          (rx_tready),
    .s_axis_twakeup         (rx_twakeup),
    .s_axis_tparity         (rx_tparity),  // 4 bits for 32-bit data

    // FUB interface
    .fub_axis5_tdata        (proc_tdata),
    .fub_axis5_tstrb        (proc_tstrb),
    .fub_axis5_tlast        (proc_tlast),
    .fub_axis5_tid          (proc_tid),
    .fub_axis5_tdest        (proc_tdest),
    .fub_axis5_tuser        (proc_tuser),
    .fub_axis5_tvalid       (proc_tvalid),
    .fub_axis5_tready       (proc_tready),
    .fub_axis5_twakeup      (proc_twakeup),
    .fub_axis5_tparity      (proc_tparity),

    // Status monitoring
    .busy                   (rx_busy),
    .parity_error           (rx_parity_err),
    .axis_clock_gating      (rx_clk_gated)
);

// Error and power monitoring
always_ff @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        parity_error_count <= '0;
        total_cycles <= '0;
        gated_cycles <= '0;
    end else begin
        // Count cycles
        total_cycles <= total_cycles + 1;
        if (rx_clk_gated)
            gated_cycles <= gated_cycles + 1;
```

```
        // Count parity errors
        if (rx_parity_err && !prev_parity_err)
            parity_error_count <= parity_error_count + 1;
        prev_parity_err <= rx_parity_err;
    end
end

// Calculate power savings
assign power_savings_pct = (gated_cycles * 100) / total_cycles;
```

3.6.3 Receive Pipeline with Power Management

```
// Network receive path with clock gating
axis5_slave_cg #(
    .AXIS_DATA_WIDTH        (512),  // Wide data path
    .AXIS_ID_WIDTH          (4),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (1),
    .CG_IDLE_COUNT_WIDTH    (6)     // Wider counter for longer idle
) u_rx_interface (
    .aclk                   (net_clk),
    .aresetn                (net_rst_n),

    // Aggressive clock gating for power
    .i_cg_enable            (1'b1),
    .i_cg_idle_count        (6'd4),  // Gate quickly after 4 idle
cycles

    // From network PHY
    .s_axis_tdata           (phy_rx_tdata),
    .s_axis_tstrb           (phy_rx_tstrb),
    .s_axis_tlast           (phy_rx_tlast),
    .s_axis_tid             (phy_rx_tid),
    .s_axis_tdest           (phy_rx_tdest),
    .s_axis_tuser           (phy_rx_tuser),
    .s_axis_tvalid          (phy_rx_tvalid),
    .s_axis_tready          (phy_rx_tready),
    .s_axis_twakeup         (phy_rx_twakeup),
    .s_axis_tparity         (phy_rx_tparity),

    // To packet processor
    .fub_axis5_tdata        (pkt_proc_tdata),
    .fub_axis5_tstrb        (pkt_proc_tstrb),
    .fub_axis5_tlast        (pkt_proc_tlast),
    .fub_axis5_tid          (pkt_proc_tid),
    .fub_axis5_tdest        (pkt_proc_tdest),
    .fub_axis5_tuser        (pkt_proc_tuser),
```

```
    .fub_axis5_tvalid       (pkt_proc_tvalid),
    .fub_axis5_tready       (pkt_proc_tready),
    .fub_axis5_twakeup      (pkt_proc_twakeup),
    .fub_axis5_tparity      (pkt_proc_tparity),

    // Status for system monitoring
    .busy                   (rx_busy),
    .parity_error           (rx_parity_err),
    .axis_clock_gating      (rx_clk_gated)
);

// Interrupt on parity error
assign parity_err_irq = rx_parity_err && !prev_rx_parity_err;
```

3.7 Design Notes

3.7.1 Clock Gating vs. Non-Gated Variant

Aspect	axis5_slave	axis5_slave_cg
Area	Smaller	+5-10% (clock gate logic)
Dynamic power	Higher	Lower (gating reduces)
Static power	Same	Same
Latency	Lower	Same (zero-cycle wake-up)
Complexity	Lower	Higher (gating control)
Use case	Always-on systems	Power-sensitive designs

When to use clock-gated variant: - Battery-powered devices - Receive paths with bursty traffic - Power budget constraints - SoC integration with power domains

3.7.2 Slave-Specific Gating Considerations

Differences from master variant: 1. **Wake-up source:** Upstream signal (`s_axis_twakeup`) 2. **Activity detection:** Input valid (`s_axis_tvalid`) 3. **Backpressure:** Backend ready affects gating

Receive path characteristics: - Often idle waiting for upstream data - Good candidate for aggressive clock gating - Wake-up from upstream provides early notification

3.7.3 Gate Transition Overhead

Each clock gate transition has overhead: - **Gate activation:** 1-2 cycles to fully gate - **Ungate activation:** 0-1 cycles to restore clock - **Dynamic power:** Gate/ungate switching consumes power

Recommendation: Set `i_cg_idle_count` high enough to amortize transition overhead, especially for high-frequency gating scenarios.

3.7.4 Parity Error Handling with Clock Gating

Parity errors detected on input side: - Check occurs before data enters skid buffer - Error flag persists regardless of clock gating state - Sticky flag ensures errors not lost during gating

Important: `parity_error` flag requires reset to clear - plan for error recovery.

3.8 Related Documentation

- **AXIS5 Slave** - Base AXIS5 slave without clock gating
- **AXIS5 Master CG** - Clock-gated master variant
- **AXIS5 Master** - Base AXIS5 master
- **AMBA Clock Gate Controller** - Clock gating controller
- **AMBA5 Power Management** - AMBA5 power features

3.9 Navigation

- **← Back to AXIS5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

4 AXIS5 Slave

Module: `axis5_slave.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

4.1 Overview

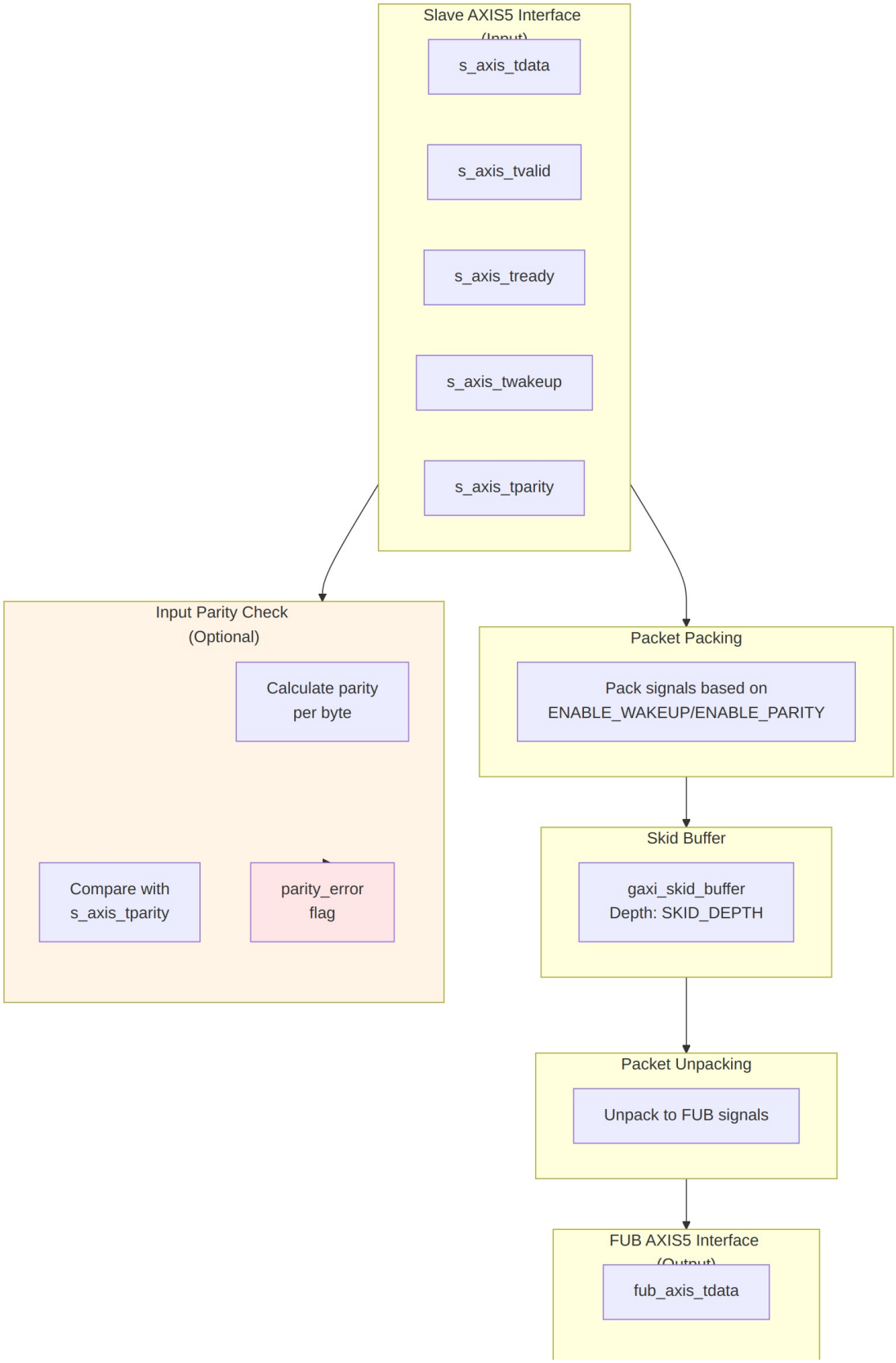
The AXIS5 Slave module implements an AXI5-Stream slave interface with AMBA5 extensions including wake-up signaling for power management and optional parity for data integrity. It accepts incoming stream data and provides buffering through an internal skid buffer for improved system performance.

4.1.1 Key Features

- Full AXI5-Stream protocol compliance
- TWAKEUP: Wake-up signaling for power management
- TPARITY: Optional parity protection (1 bit per byte)
- Internal skid buffer for backpressure handling
- Configurable data, ID, destination, and user signal widths
- Parity error detection and reporting
- Busy status indication

4.1.2 AXIS5 Extensions Over AXIS4

Feature	AXIS4	AXIS5
Wake-up signal	None	TWAKEUP (configurable)
Data parity	None	TPARITY (1 bit per byte, configurable)
Parity error detection	None	Built-in error flag


Slave AXIS5 Interface
s_axis_tdata
s_axis_tvalid
s_axis_tready
s_axis_twakeup
s_axis_tparity
Input Parity Check
(Optional)
Calculate parity
per byte
Compare with
s_axis_tparity
parity_error
flag
Packet Packing
Pack signals based on
ENABLE_WAKEUP/ENABLE_PARITY
Skid Buffer
gaxi_skid_buffer
Depth: SKID_DEPTH
Packet Unpacking
Unpack to FUB signals
FUB AXIS5 Interface
fub_axis_tdata

4.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable TPARITY signal (1=enabled)
DW	int	AXIS_DATA_WIDTH	Data width short name (calculated)
IW	int	AXIS_ID_WIDTH	ID width short name (calculated)
DESTW	int	AXIS_DEST_WIDTH	DEST width short name (calculated)
UW	int	AXIS_USER_WIDTH	USER width short name (calculated)
SW	int	DW/8	Strobe width in bytes (calculated)
PW	int	SW	Parity width - 1 bit per byte (calculated)

4.3 Ports

4.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS clock
aresetn	1	Input	AXIS active-low asynchronous reset

4.3.2 Slave AXIS5 Interface (Input Side)

Port	Width	Direction	Description
s_axis_tdata	DW	Input	Transfer data from upstream
s_axis_tstrb	SW	Input	Transfer byte strobes
s_axis_tlast	1	Input	Last transfer in packet
s_axis_tid	IW_WIDTH	Input	Transfer ID (optional)
s_axis_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
s_axis_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
s_axis_tvalid	1	Input	Transfer valid from upstream
s_axis_tready	1	Output	Transfer ready (skid buffer not full)
s_axis_twakeup	1	Input	Wake-up signal (AXIS5 extension)
s_axis_tparity	PW_WIDTH	Input	Data parity per byte (AXIS5 extension)

4.3.3 FUB AXIS5 Interface (Output Side to Backend)

Port	Width	Direction	Description
fub_axis_tdata	DW	Output	Transfer data to backend

Port	Width	Direction	Description
fub_axis_tstrb	SW	Output	Transfer byte strobes
fub_axis_tlast	1	Output	Last transfer in packet
fub_axis_tid	IW_WIDTH	Output	Transfer ID (optional)
fub_axis_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
fub_axis_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
fub_axis_tvalid	1	Output	Transfer valid to backend
fub_axis_tready	1	Input	Transfer ready from backend
fub_axis_twakeup	1	Output	Wake-up signal (AXIS5 extension)
fub_axis_tparity	PW_WIDTH	Output	Data parity per byte (AXIS5 extension)

4.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Parity error detected on input (sticky flag)

4.4 Functionality

4.4.1 Skid Buffer Operation

The module uses an internal `gaxi_skid_buffer` to: - Accept incoming transfers even when backend is not ready - Prevent backpressure to upstream - Provide registered outputs for timing closure - Track buffer occupancy via `busy` signal

4.4.2 Packet Packing/Unpacking

Conditional packing based on configuration:

```
// Full feature set (ENABLE_WAKEUP=1, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup, tparity}

// Wake-up only (ENABLE_WAKEUP=1, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup}

// Parity only (ENABLE_WAKEUP=0, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, tparity}

// Base AXIS4 (ENABLE_WAKEUP=0, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser}
```

4.4.3 Parity Checking (Optional)

When `ENABLE_PARITY=1`: 1. Calculate odd parity for each input data byte: `parity[i] = ^s_axis_tdata[i*8 +: 8]` 2. Compare calculated parity with received `s_axis_tparity` 3. Set `parity_error` flag on mismatch (sticky, cleared by reset) 4. Parity check occurs on valid input transfers (`s_axis_tvalid && s_axis_tready`)

Note: Parity checking is performed on the **input side** to detect errors as early as possible.

4.4.4 Busy Signal

The `busy` output indicates: - Input side has valid data (`s_axis_tvalid`) - Skid buffer contains data (`int_t_count > 0`)

4.5 Timing Diagrams

4.5.1 Basic Receive with Wake-up

```
TODO: Wavedrom timing diagram showing:
- aclk
- s_axis_tvalid/tready
- s_axis_tdata
- s_axis_tlast
- s_axis_twakeup (AXIS5 extension)
- fub_axis_tvalid/tready
- fub_axis_tdata
- fub_axis_twakeup
- busy
```

4.5.2 Receive with Parity Error

```
TODO: Wavedrom timing diagram showing:
- aclk
- s_axis_tdata
- s_axis_tparity (received)
- calculated_parity
- parity_mismatch
- parity_error (sticky flag set)
```

4.5.3 Skid Buffer Backpressure from Backend

```
TODO: Wavedrom timing diagram showing:
- aclk
- s_axis_tvalid/tready
- fub_axis_tvalid/tready (backend blocked)
- int_t_count (buffer fill level)
- busy
```

4.6 Usage Example

4.6.1 Basic Configuration

```
axis5_slave #(
    .SKID_DEPTH       (4),
    .AXIS_DATA_WIDTH  (64),
    .AXIS_ID_WIDTH    (8),
    .AXIS_DEST_WIDTH  (4),
    .AXIS_USER_WIDTH  (1),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (0)
) u_axis5_slave (
    .aclk               (axis_clk),
    .aresetn            (axis_rst_n),

    // Slave interface (from upstream)
    .s_axis_tdata       (s_axis_tdata),
    .s_axis_tstrb       (s_axis_tstrb),
    .s_axis_tlast       (s_axis_tlast),
    .s_axis_tid         (s_axis_tid),
    .s_axis_tdest       (s_axis_tdest),
    .s_axis_tuser       (s_axis_tuser),
    .s_axis_tvalid      (s_axis_tvalid),
    .s_axis_tready      (s_axis_tready),
    .s_axis_twakeup     (s_axis_twakeup),
    .s_axis_tparity     (8'h00),  // Not used when ENABLE_PARITY=0
```

```
    // FUB interface (to backend)
    .fub_axis_tdata      (fub_tdata),
    .fub_axis_tstrb      (fub_tstrb),
    .fub_axis_tlast      (fub_tlast),
    .fub_axis_tid        (fub_tid),
    .fub_axis_tdest      (fub_tdest),
    .fub_axis_tuser      (fub_tuser),
    .fub_axis_tvalid     (fub_tvalid),
    .fub_axis_tready     (fub_tready),
    .fub_axis_twakeup    (fub_twakeup),
    .fub_axis_tparity    (),  // Not used when ENABLE_PARITY=0

    // Status
    .busy                (axis_busy),
    .parity_error        ()  // Not used when ENABLE_PARITY=0
);
```

4.6.2 With Parity Protection and Error Handling

```
axis5_slave #(
    .AXIS_DATA_WIDTH  (32),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (1)  // Enable parity checking on input
) u_axis5_slave_parity (
    .aclk                (axis_clk),
    .aresetn             (axis_rst_n),

    // Slave interface with parity
    .s_axis_tdata        (s_axis_tdata),
    .s_axis_tstrb        (s_axis_tstrb),
    .s_axis_tlast        (s_axis_tlast),
    .s_axis_tid          (s_axis_tid),
    .s_axis_tdest        (s_axis_tdest),
    .s_axis_tuser        (s_axis_tuser),
    .s_axis_tvalid       (s_axis_tvalid),
    .s_axis_tready       (s_axis_tready),
    .s_axis_twakeup      (s_axis_twakeup),
    .s_axis_tparity      (s_axis_tparity),  // 4 bits for 32-bit data

    // FUB interface
    .fub_axis_tdata      (fub_tdata),
    .fub_axis_tstrb      (fub_tstrb),
    .fub_axis_tlast      (fub_tlast),
    .fub_axis_tid        (fub_tid),
    .fub_axis_tdest      (fub_tdest),
    .fub_axis_tuser      (fub_tuser),
    .fub_axis_tvalid     (fub_tvalid),
    .fub_axis_tready     (fub_tready),
```

```
    .fub_axis_twakeup    (fub_twakeup),
    .fub_axis_tparity    (fub_tparity),

    // Status - monitor parity errors
    .busy                (axis_busy),
    .parity_error        (axis_parity_err)  // Sticky error flag
);

// Error handling and logging
always_ff @(posedge axis_clk or negedge axis_rst_n) begin
    if (!axis_rst_n) begin
        error_count <= '0;
        prev_parity_err <= 1'b0;
    end else begin
        prev_parity_err <= axis_parity_err;
        // Count rising edges (new errors)
        if (axis_parity_err && !prev_parity_err) begin
            error_count <= error_count + 1;
            // Log error or trigger interrupt
            $error("AXIS5 Slave: Parity error detected at time %t",
$time);
        end
    end
end
```

4.6.3 Integration with Processing Pipeline

```
// AXIS5 slave receives data from network/upstream
axis5_slave #(
    .AXIS_DATA_WIDTH  (512),  // Wide data path
    .AXIS_ID_WIDTH    (4),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (1)
) u_rx_slave (
    .aclk              (sys_clk),
    .aresetn           (sys_rst_n),
    .s_axis_tdata      (rx_tdata),
    .s_axis_tstrb      (rx_tstrb),
    .s_axis_tlast      (rx_tlast),
    .s_axis_tid        (rx_tid),
    .s_axis_tdest      (rx_tdest),
    .s_axis_tuser      (rx_tuser),
    .s_axis_tvalid     (rx_tvalid),
    .s_axis_tready     (rx_tready),
    .s_axis_twakeup    (rx_twakeup),
    .s_axis_tparity    (rx_tparity),
    // To processing pipeline
    .fub_axis_tdata    (proc_tdata),
```

```
    .fub_axis_tstrb    (proc_tstrb),
    .fub_axis_tlast    (proc_tlast),
    .fub_axis_tid      (proc_tid),
    .fub_axis_tdest    (proc_tdest),
    .fub_axis_tuser    (proc_tuser),
    .fub_axis_tvalid   (proc_tvalid),
    .fub_axis_tready   (proc_tready),
    .fub_axis_twakeup  (proc_twakeup),
    .fub_axis_tparity  (proc_tparity),
    .busy              (rx_busy),
    .parity_error      (rx_parity_err)
);

// Connect to processing module
my_data_processor u_processor (
    .clk         (sys_clk),
    .rst_n       (sys_rst_n),
    .in_tdata    (proc_tdata),
    .in_tvalid   (proc_tvalid),
    .in_tready   (proc_tready),
    // ... other signals
);
```

4.7 Design Notes

4.7.1 AXIS5 vs AXIS4 Differences

Feature	AXIS4	AXIS5
Wake-up signal	Not supported	TWAKEUP (optional)
Data parity	Not supported	TPARITY per byte (optional)
Power management	Limited	Enhanced via TWAKEUP
Data integrity	CRC/checksum in TUSER	Built-in parity option

4.7.2 Parity Check Location

Key difference from AXIS5 Master: - **Slave:** Parity checked on **input** (`s_axis_tdata` vs `s_axis_tparity`) - **Master:** Parity checked on **output** (`m_axis_tdata` vs `m_axis_tparity`)

This allows early detection of transmission errors at the receiving side.

4.7.3 Skid Buffer Sizing

- **Typical depth:** 4-8 entries
- Deeper buffers:
  - Absorb longer backend stalls
  - Increase area and latency
- Shallower buffers:
  - Lower latency
  - May cause upstream backpressure

Recommendation: Use depth 4 for most applications, increase if backend frequently stalls.

4.7.4 Parity Implementation

When `ENABLE_PARITY=1`: - **Overhead:** 1 bit per data byte (12.5% for 8-bit bytes) - **Detection:** Single-bit errors only (odd parity) - **Correction:** None - error flag only - **Use case:** Low-cost error detection in reliable environments

For stronger protection, use: - CRC in TUSER field - ECC (external module) - End-to-end checksums at packet level

4.7.5 Error Handling Strategy

The `parity_error` flag is **sticky** (latches high until reset): - Simplifies error detection (level-sensitive interrupt) - Software must clear by resetting module - For edge-sensitive interrupts, external logic must detect rising edge

Alternative: Modify module to add `parity_error_clear` input if needed.

4.8 Related Documentation

- **AXIS5 Master** - AXIS5 master interface (transmit side)
- **AXIS5 Slave CG** - Clock-gated variant with power management
- **AXIS5 Master CG** - Clock-gated master variant
- **AXIS4 Slave** - AXIS4 version for comparison
- **AMBA5 Overview** - AMBA5 specifications and extensions

4.9 Navigation

- **← Back to AXIS5 Index**
- **← Back to RTLAmba Index**

- ← Back to Main Documentation Index

5 AXIS5 (AXI5-Stream) Modules

Location: `rtl/amba/axis5/` **Test Location:** `val/amba/` **Status:** Production Ready

---

5.1 Overview

The AXIS5 subsystem provides a complete implementation of the ARM AMBA 5 AXI-Stream protocol, including masters, slaves, and clock-gated variants for high-throughput streaming data applications.

AXI5-Stream extends AXI4-Stream with enhancements for modern streaming data applications, including improved signaling for wake-up, additional user signals, and enhanced flow control mechanisms.

5.2 AMBA4 vs AMBA5 Comparison

Feature	AXI4-Stream	AXI5-Stream
Basic Protocol	TVALID/TREADY handshake	TVALID/TREADY handshake
Wake-up	Not supported	TWAKEUP signal
User Signal Width	TUSER (variable)	TUSER (extended)
Data Poisoning	Not supported	TPOISON for error marking
Chunking	Not supported	TCHUNKEN for segmentation
Side-band Signals	TID, TDEST, TUSER	Enhanced TID, TDEST, TUSER

5.3 Module Categories

5.3.1 Stream Master Components

Module	Description	Documentation	Status
axis5_maste	AXI5-Stream master	axis5_master.md	Documented

Module	Description	Documentation	Status
r	for high-throughput data streaming		
axis5_master_cg	Clock-gated AXI5-Stream master	axis5_master_cg.md	Documented

5.3.2 Stream Slave Components

Module	Description	Documentation	Status
axis5_slave	AXI5-Stream slave for data reception	axis5_slave.md	Documented
axis5_slave_cg	Clock-gated AXI5-Stream slave	axis5_slave_cg.md	Documented

5.4 Key Features

5.4.1 AXI5-Stream Protocol Support

- **Full AXI5-Stream Compliance:** Complete AMBA 5 streaming protocol
- **Flow Control:** TVALID/TREADY handshaking with backpressure
- **Packet Boundaries:** TLAST for packet/frame demarcation
- **Byte Qualification:** TSTRB and TKEEP for byte-level control
- **Side-band Data:** TID, TDEST, TUSER for routing and metadata

5.4.2 Enhanced AXI5 Features

- **Wake-up Signaling:** TWAKEUP for low-power state exit
- **Data Poisoning:** TPOISON for error propagation in pipelines
- **Extended User Signals:** Wider TUSER for additional metadata

5.4.3 Power Management

- **Clock Gating:** Per-module clock gating for power reduction
- **Wake-up Support:** TWAKEUP integration for power management
- **Idle Detection:** Automatic clock gate when stream is idle

5.4.4 Performance

- **Zero-Latency Option:** Combinational pass-through mode
- **Registered Mode:** Optional pipeline stages for timing

- **Configurable Widths:** Flexible data and signal widths

5.5 Quick Start

5.5.1 Using AXI5-Stream Master

```
axis5_master #(
    .SKID_DEPTH_T(4),
    .AXIS_DATA_WIDTH(128),
    .AXIS_ID_WIDTH(8),
    .AXIS_DEST_WIDTH(4),
    .AXIS_USER_WIDTH(8)
) u_axis5_master (
    .aclk               (clk),
    .aresetn            (resetn),

    // FUB streaming interface (from internal logic)
    .fub_axis_tdata     (fub_tdata),
    .fub_axis_tstrb     (fub_tstrb),
    .fub_axis_tkeep     (fub_tkeep),
    .fub_axis_tlast     (fub_tlast),
    .fub_axis_tid       (fub_tid),
    .fub_axis_tdest     (fub_tdest),
    .fub_axis_tuser     (fub_tuser),
    .fub_axis_tvalid    (fub_tvalid),
    .fub_axis_tready    (fub_tready),

    // AXI5-Stream master interface
    .m_axis_tdata       (m_tdata),
    .m_axis_tstrb       (m_tstrb),
    .m_axis_tkeep       (m_tkeep),
    .m_axis_tlast       (m_tlast),
    .m_axis_tid         (m_tid),
    .m_axis_tdest       (m_tdest),
    .m_axis_tuser       (m_tuser),
    .m_axis_twakeup     (m_twakeup),
    .m_axis_tvalid      (m_tvalid),
    .m_axis_tready      (m_tready)
);
```

5.5.2 Using AXI5-Stream Slave

```
axis5_slave #(
    .SKID_DEPTH_T(4),
    .AXIS_DATA_WIDTH(128),
    .AXIS_ID_WIDTH(8),
    .AXIS_DEST_WIDTH(4),
```

```
    .AXIS_USER_WIDTH(8)
) u_axis5_slave (
    .aclk               (clk),
    .aresetn            (resetn),

    // AXI5-Stream slave interface
    .s_axis_tdata       (s_tdata),
    .s_axis_tstrb       (s_tstrb),
    .s_axis_tkeep       (s_tkeep),
    .s_axis_tlast       (s_tlast),
    .s_axis_tid         (s_tid),
    .s_axis_tdest       (s_tdest),
    .s_axis_tuser       (s_tuser),
    .s_axis_twakeup     (s_twakeup),
    .s_axis_tvalid      (s_tvalid),
    .s_axis_tready      (s_tready),

    // FUB streaming interface (to internal logic)
    .fub_axis_tdata     (fub_tdata),
    .fub_axis_tstrb     (fub_tstrb),
    .fub_axis_tkeep     (fub_tkeep),
    .fub_axis_tlast     (fub_tlast),
    .fub_axis_tid       (fub_tid),
    .fub_axis_tdest     (fub_tdest),
    .fub_axis_tuser     (fub_tuser),
    .fub_axis_tvalid    (fub_tvalid),
    .fub_axis_tready    (fub_tready)
);
```

5.5.3 Clock-Gated Streaming

```
// Clock-gated master for power efficiency
axis5_master_cg #(
    .AXIS_DATA_WIDTH(128)
) u_axis5_master_cg (
    .aclk               (clk),
    .aresetn            (resetn),

    // Clock gating control
    .cg_enable          (stream_active),
    .cg_test_enable     (scan_test_mode),
    .gated_clk          (stream_gated_clk),

    // Streaming interfaces
    .fub_axis_*         (...),
    .m_axis_*           (...)
);
```

5.6 Testing

All AXIS5 modules are verified using CocoTB-based testbenches located in `val/amba/`:

```
# Run all AXIS5 tests
pytest val/amba/test_axis5*.py -v

# Run specific module tests
pytest val/amba/test_axis5_master.py -v
pytest val/amba/test_axis5_slave.py -v
```

5.7 Protocol Details

5.7.1 AXI5-Stream Signal Descriptions

Signal	Direction	Description
ACLK	Input	Stream clock
ARESETn	Input	Active-low reset
TDATA	Master to Slave	Stream data payload
TSTRB	Master to Slave	Byte strobes (data vs. position)
TKEEP	Master to Slave	Byte qualifiers (valid bytes)
TLAST	Master to Slave	End of packet/frame indicator
TID	Master to Slave	Stream identifier
TDEST	Master to Slave	Destination routing information
TUSER	Master to Slave	User-defined sideband data
TWAKEUP	Master to Slave	Wake-up signal (AXI5)
TVALID	Master to Slave	Data valid indicator
TREADY	Slave to Master	Ready to accept data

5.7.2 Flow Control

AXI5-Stream uses simple valid/ready handshaking:

1. **Data Transfer:** Occurs when TVALID and TREADY are both high
2. **Backpressure:** Slave deasserts TREADY to pause stream
3. **Source Stall:** Master deasserts TVALID when no data available
4. **Packet End:** TLAST indicates final beat of packet/frame

5.7.3 Byte Qualification

TKEEP	TSTRB	Meaning
1	1	Data byte
1	0	Position byte (placeholder)
0	0	Null byte (not transmitted)
0	1	Reserved

5.8 Design Notes

5.8.1 FUB Interface Pattern

All AXIS5 modules use the FUB (Functional Unit Block) interface pattern: - **fub_axis_*** signals connect to internal logic - **m_axis_*** or **s_axis_*** signals connect to external stream bus - Skid buffers between FUB and external interfaces for timing closure

5.8.2 Migration from AXI4-Stream

AXIS5 modules are backward compatible with AXI4-Stream: - Core protocol unchanged (TVALID/TREADY handshake) - TWAKEUP can be tied to 0 for always-awake operation - TUSER width can match AXI4-Stream configurations - New signals are optional additions

5.8.3 Streaming Pipeline Integration

AXIS5 modules integrate seamlessly into streaming pipelines:

```
// Streaming data processing pipeline
axis5_master u_input (
    .m_axis_*(stage1_axis)
);

// Processing stage
```

```
processing_block u_process (
    .s_axis_*(stage1_axis),
    .m_axis_*(stage2_axis)
);

// Clock domain crossing
gaxi_fifo_async u_cdc (
    .s_*(stage2_axis),
    .m_*(stage3_axis)
);

axis5_slave u_output (
    .s_axis_*(stage3_axis)
);
```

5.9 Performance Characteristics

Metric	Typical Value
Maximum Frequency	600+ MHz
Latency (skid buffer)	0-2 clock cycles
Throughput (128-bit, 600 MHz)	76.8 GB/s
Backpressure Response	1 clock cycle

5.10 Related Documentation

- **AXIS4 Modules** - AXI4-Stream components
- **AXI5 Modules** - AXI5 protocol components
- **APB5 Modules** - APB5 protocol components
- **GAXI Modules** - Generic AXI utilities (FIFOs, CDC)

5.11 References

5.11.1 Specifications

- ARM AMBA 5 AXI-Stream Protocol Specification
- ARM AMBA AXI4-Stream Protocol Specification

5.11.2 Source Code

- RTL: `rtl/amba/axis5/`
- Tests: `val/amba/test_axis5*.py`
- Framework: `bin/TBClasses/components/axis4/`

Last Updated: 2025-12-26

5.12 Navigation

- **Back to RTLAmba Index**
- **Back to Main Documentation Index**